Tasks on docker-03

1. Create a image from running container.

Implementation steps

1. List the running containers.

docker ps

2. Select the container whose current configuration and changes you want to save as an image.

3. Create an image from the running container using docker commit.

docker commit <container_name> custom-image:v1

4. Verify that the image was created.

docker images

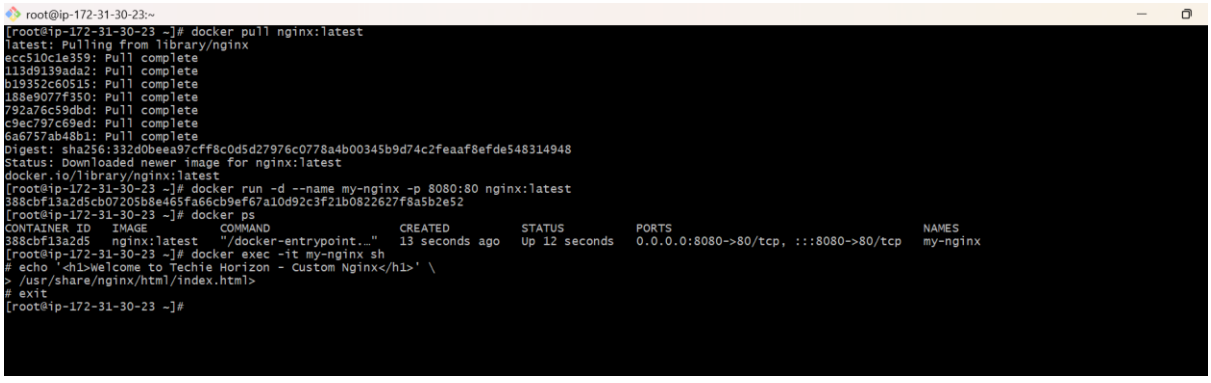
5. Create a new container from the generated image.

docker run -d --name custom-container custom-image:v1

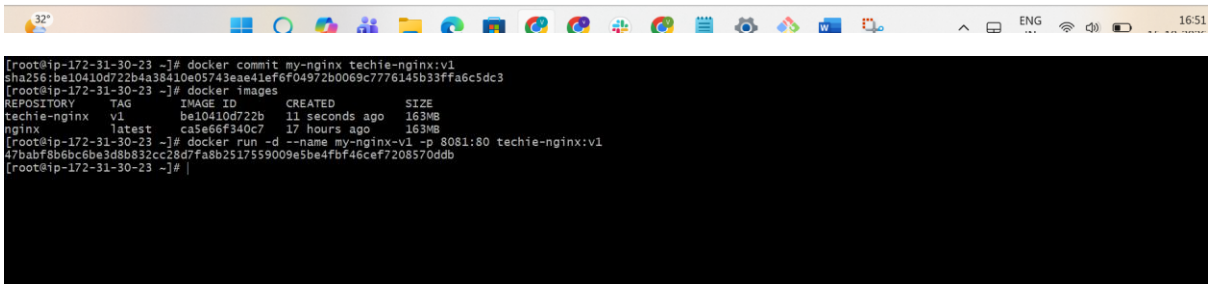
6. Verify the new container.

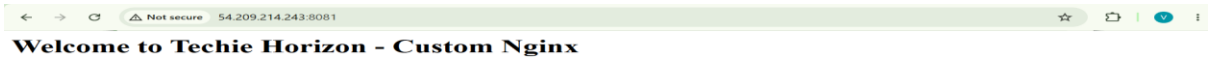
docker ps

Result: A Docker image was created from an existing container and used to launch a new container.









2. Copy image from local machine to docker server and load the image.

Implementations steps

1. List the available images on the source machine.

docker images

2. Save the required image to a TAR archive.

docker save -o custom-image.tar custom-image:v1

3. Copy the archive to the destination Docker server using scp.

scp -i linux-testing.pem custom-image.tar ec2-user@<SERVER_PUBLIC_IP>:/home/ec2-user/

Replace the key filename and server IP with the actual values.

4. Connect to the destination server.

ssh -i linux-testing.pem ec2-user@<SERVER_PUBLIC_IP>

5. Load the image from the TAR archive.

docker load -i /home/ec2-user/custom-image.tar

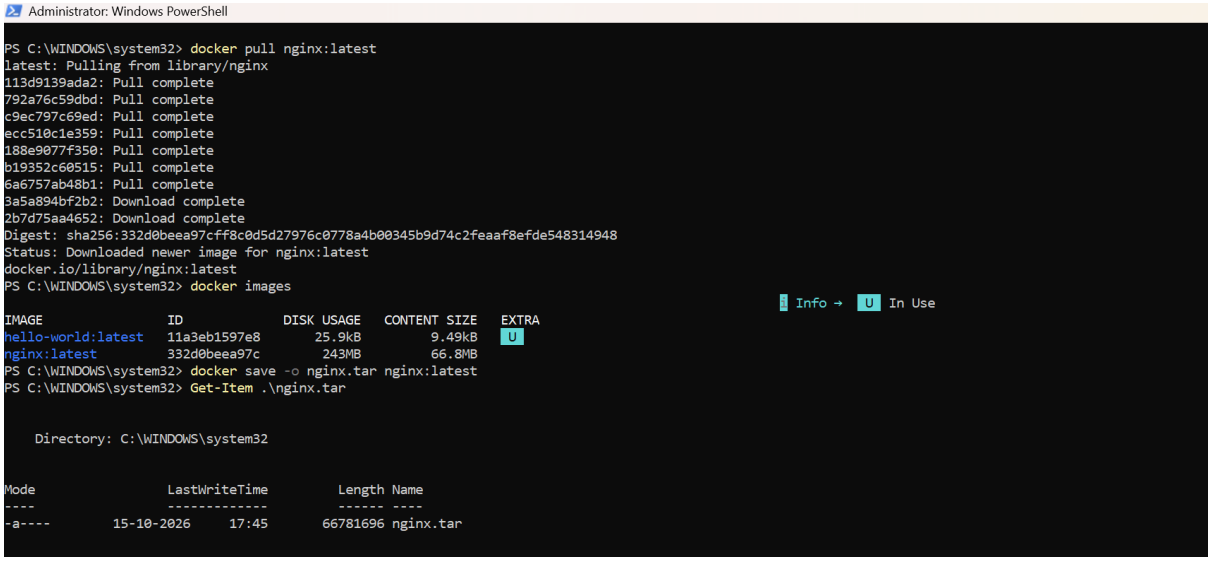
6. Verify that the image exists on the destination server.

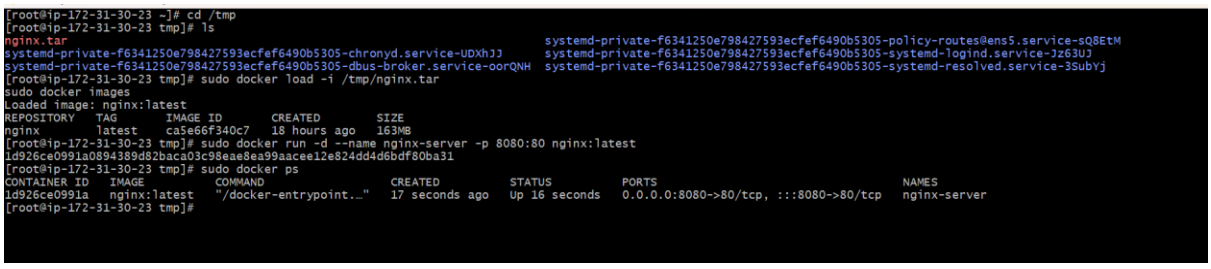
docker images

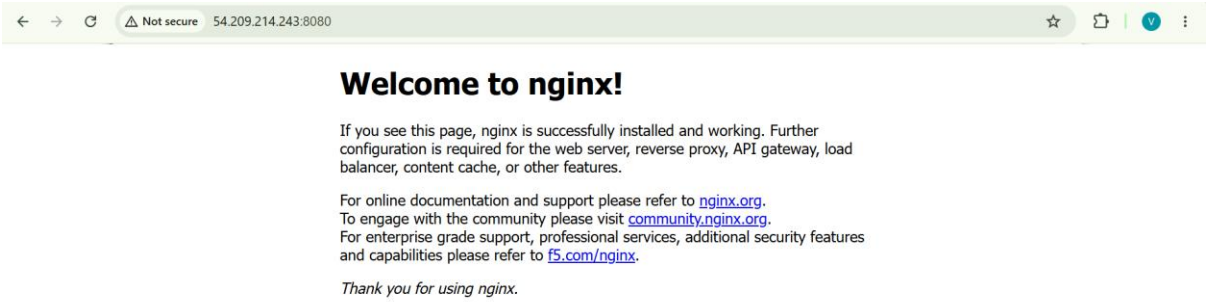
7. Run a container from the loaded image if required.

docker run -d --name copied-image-container custom-image:v1

Result: The Docker image was transferred to another server and loaded without pulling it from a registry.







3. Create Docker image using alpine and customize with tomcat.

Implementation steps

1. Create a project directory.
2. mkdir alpine-tomcat

cd alpine-tomcat

3. Create a file named Dockerfile.
4. FROM alpine:3.22
5.
6. RUN apk add --no-cache openjdk17-jre wget tar
7.
8. ENV CATALINA_HOME=/opt/tomcat
9. ENV PATH=$CATALINA_HOME/bin:$PATH
10.
11. RUN wget -q https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.46/bin/apache-tomcat-10.1.46.tar.gz \
12. -O /tmp/tomcat.tar.gz \
13. && mkdir -p /opt/tomcat \
14. && tar -xzf /tmp/tomcat.tar.gz \
15. --strip-components=1 -C /opt/tomcat \
16. && rm /tmp/tomcat.tar.gz \
17. && chmod +x /opt/tomcat/bin/*.sh
18.
19. EXPOSE 8080
20.

CMD ["catalina.sh", "run"]

21. Build the image.

docker build -t alpine-tomcat:v1 .

22. Verify the image.

docker images

23. Run a container and map port 8081 on the host to port 8080 in the container.

docker run -d --name alpine-tomcat-app -p 8081:8080 alpine-tomcat:v1

24. Check the container status and logs.

25. docker ps

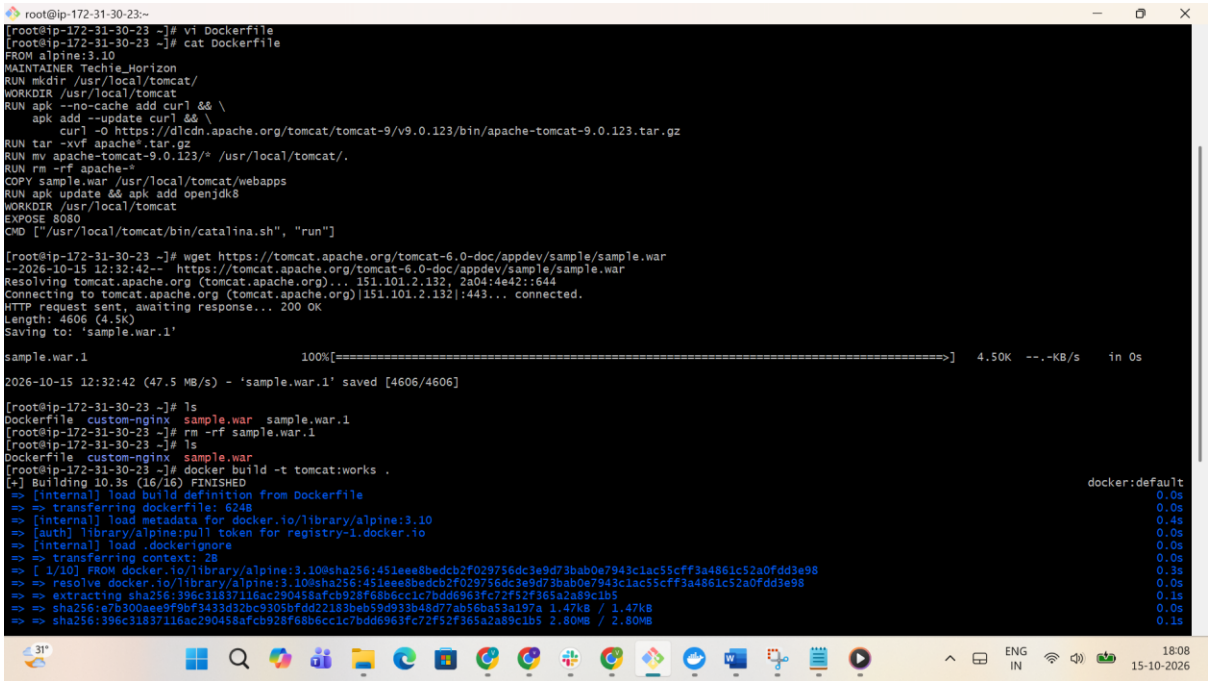
docker logs alpine-tomcat-app

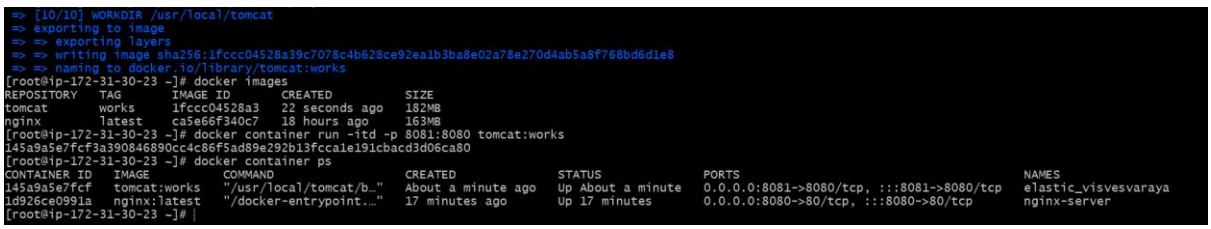
26. Open the Tomcat URL in a browser.

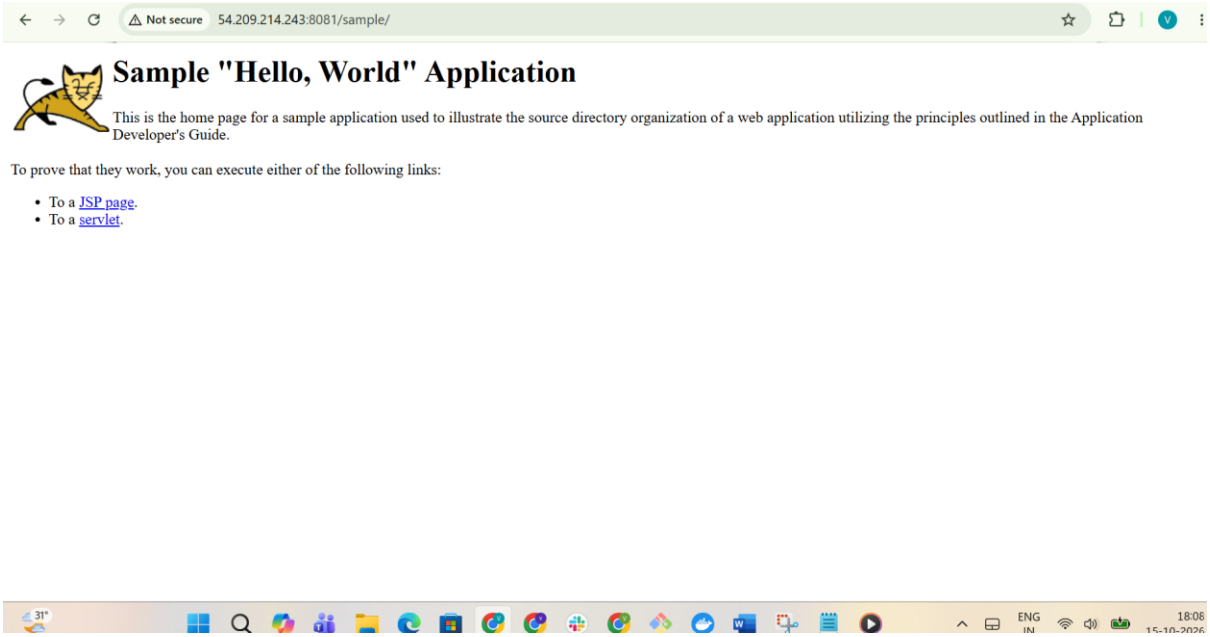
http://<SERVER_PUBLIC_IP>:8081

Ensure the server security group permits inbound TCP port 8081 from your IP.

Result: A customized Docker image was created using Alpine Linux as the base image and Apache Tomcat as the application server.







4. Create single stage and multi stage docker file using this source code: https://github.com/betawins/multi-stage-example.git

Implementation steps

A. Multi-stage Dockerfile

1. Create a file named Dockerfile.multi in the same project directory.

```
# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-8 AS builder
WORKDIR /app/source
COPY . .
RUN mvn -Dmaven.repo.local=/tmp/m2 clean package -DskipTests

# Stage 2: Run the application
FROM eclipse-temurin:8-jre-alpine
WORKDIR /app
COPY --from=builder /app/source/target/*.jar /app/app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "/app/app.jar"]
```

2. Build the multi-stage image.

```
docker build -f Dockerfile.multi -t multistage:v1 .
```

3. Verify the image.

docker images

4. Run the container.

docker run -d --name multi-app -p 8082:8080 multistage:v1

5. Verify the running container.

docker ps

docker logs multi-app

6. Open the application in a browser using http://<SERVER_PUBLIC_IP>:8082, after allowing inbound TCP port 8082 in the EC2 security group.

7. Compare the images using docker images. The multi-stage build separates compilation tools from the runtime image, which can reduce the final image size.

```
[root@ip-172-31-30-23 multi-stage-example]# cat Dockerfile.multi
# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-8 AS builder

WORKDIR /app/source

COPY . .

RUN mvn -Dmaven.repo.local=/tmp/m2 clean package -DskipTests

# Stage 2: Run the application
FROM eclipse-temurin:8-jre-alpine

WORKDIR /app

COPY --from=builder /app/source/target/*.jar /app/app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "/app/app.jar"]
[root@ip-172-31-30-23 multi-stage-example]#
```

```
EOF
[root@ip-172-31-30-23 multi-stage-example]# docker build --no-cache -f Dockerfile.multi -t multistage:v1 .
[+] Building 26.7s (13/13) FINISHED                                                
```



B. Single-stage Dockerfile

1. Clone the repository.
2. git clone https://github.com/betawins/multi-stage-example.git

   cd multi-stage-example
3. Create a file named Dockerfile.single.

   FROM maven:3.9-eclipse-temurin-8

   WORKDIR /app/source

   COPY . .

   RUN mvn -Dmaven.repo.local=/tmp/m2 clean package -DskipTests

   EXPOSE 8080

   ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "/app/source/target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
4. Build the image.

   docker build -f Dockerfile.single -t singlestage:v1 .
5. Verify the image.

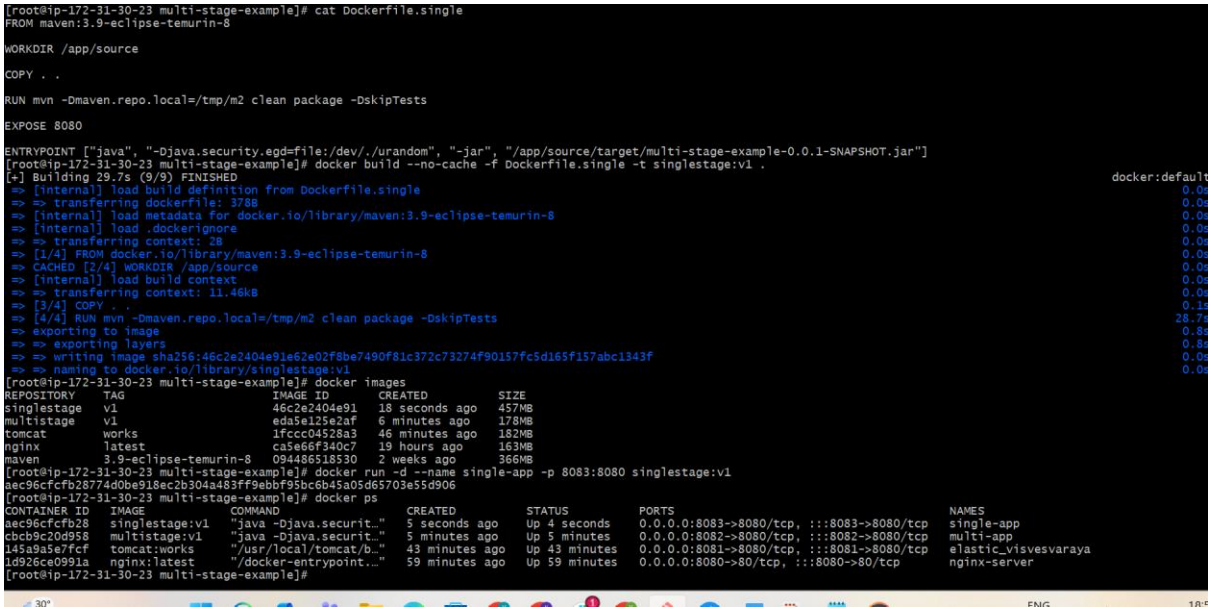
   docker images
6. Run the container.

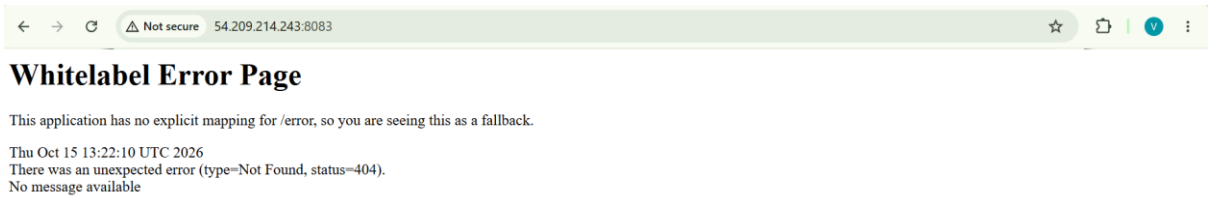
   docker run -d --name single-app -p 8083:8080 singlestage:v1
7. Verify the container and application logs.

   docker ps

   docker logs single-app

Result: Single-stage and multi-stage Docker images were created and used to run the Java application.





5. Install docker compose and execute sample application.

Implementations steps

A. Install Docker Compose on Amazon Linux

1. Check the machine architecture.

uname -m

2. For an x86_64 EC2 instance, download the Docker Compose standalone CLI plugin binary.
3. mkdir -p /usr/local/lib/docker/cli-plugins
4.
5. curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
6. -o /usr/local/lib/docker/cli-plugins/docker-compose
7.

chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

For an aarch64 instance, use docker-compose-linux-aarch64 in the download URL instead.

8. Verify the installation.

docker compose version

B. Create a sample WordPress application

1. Create a project directory.

   mkdir wordpress-compose

   cd wordpress-compose

2. Create a file named compose.yaml.

```
services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: change_this_root_password
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: change_this_app_password
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    restart: always
    ports:
      - "8000:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: change_this_app_password
      WORDPRESS_DB_NAME: wordpress
volumes:
```

db_data:

Replace the sample passwords with strong values before use. This example uses MySQL 5.7 for the lab; it is an older release and is not recommended for a new production deployment.

3. Validate the Compose file.

docker compose config

4. Start the application.

docker compose up -d

5. Verify the services.

   docker compose ps

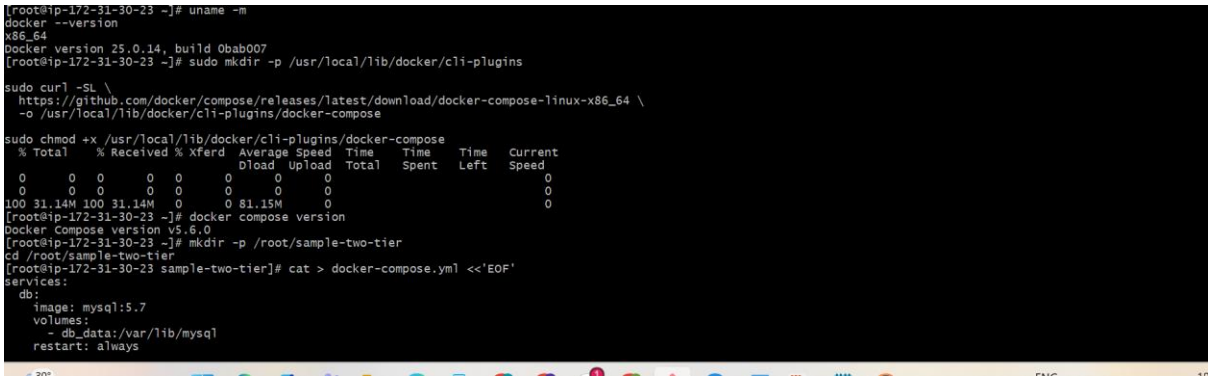
docker compose logs

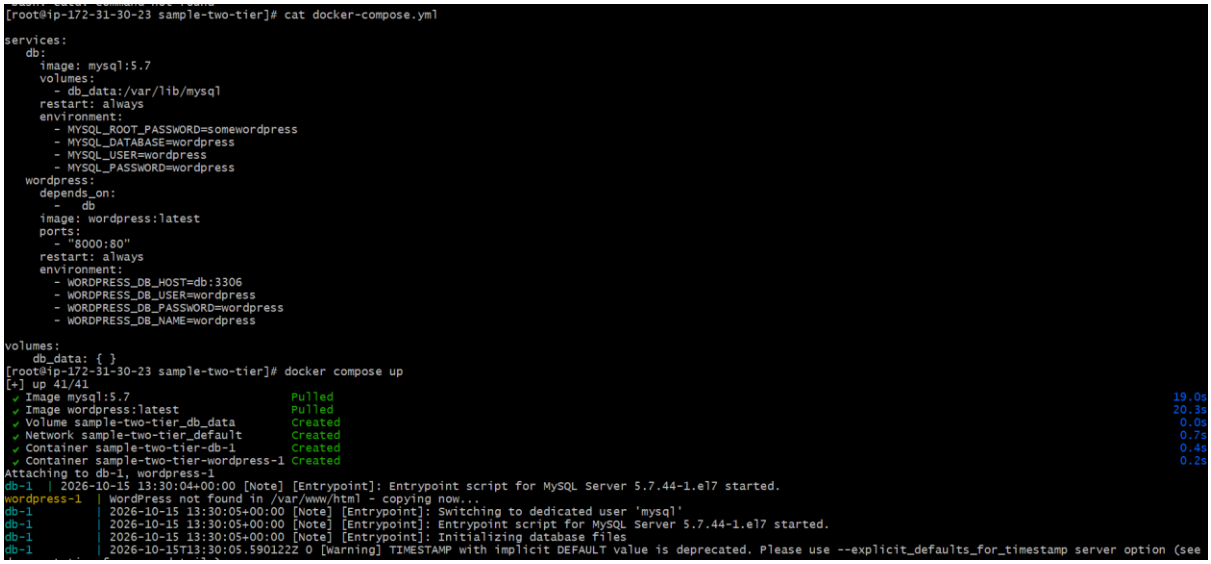
6. Open the application at http://<SERVER_PUBLIC_IP>:8000. Allow inbound TCP port 8000 in the EC2 security group if required.

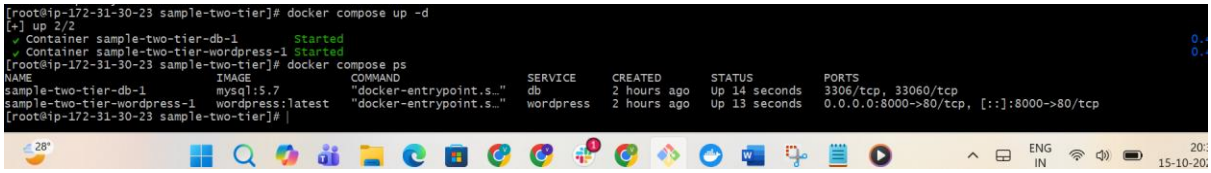
7. Stop the application when finished.

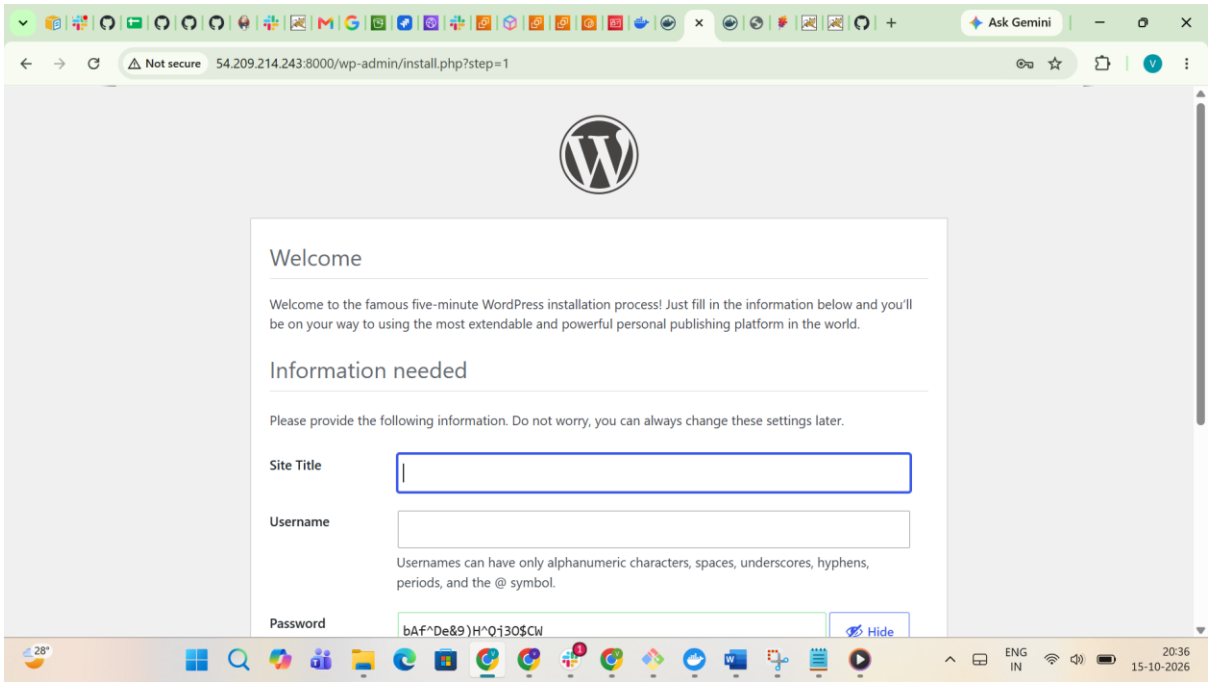
docker compose down

Result: Docker Compose was installed and used to run a multi-container application consisting of WordPress and MySQL.









6. Implement solution to scan images when pushed to docker registry.

Implementation steps

For Docker Hub, Docker Scout can analyze images for software vulnerabilities and maintain image analysis results for enabled repositories.

1. Sign in to Docker Hub.
2. Open the Docker Scout dashboard and enable image analysis for the target repository, where the feature is available for your account.

3. Sign in to Docker on the machine that builds the image.

docker login

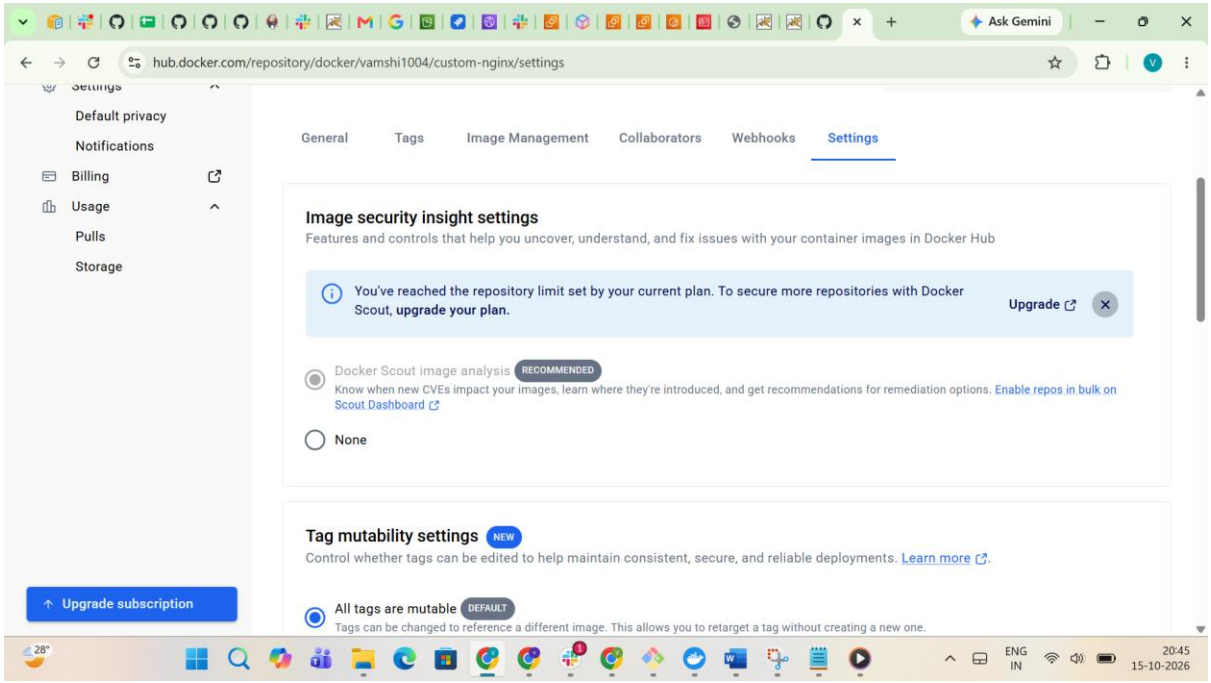
4. Build and tag the image.

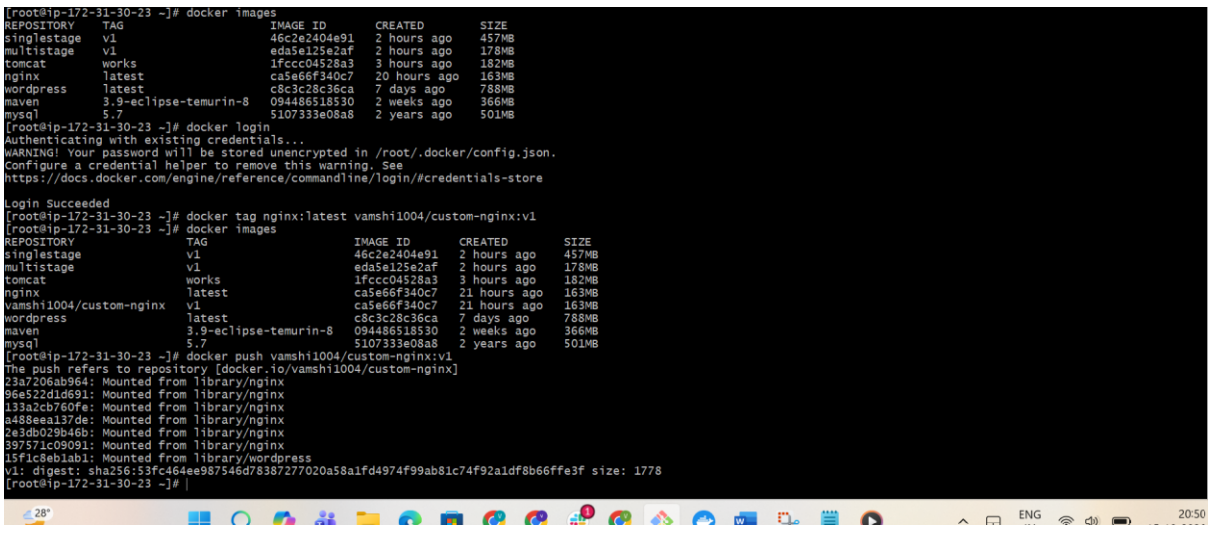
docker build -t vamshi1004/custom-nginx:v1.

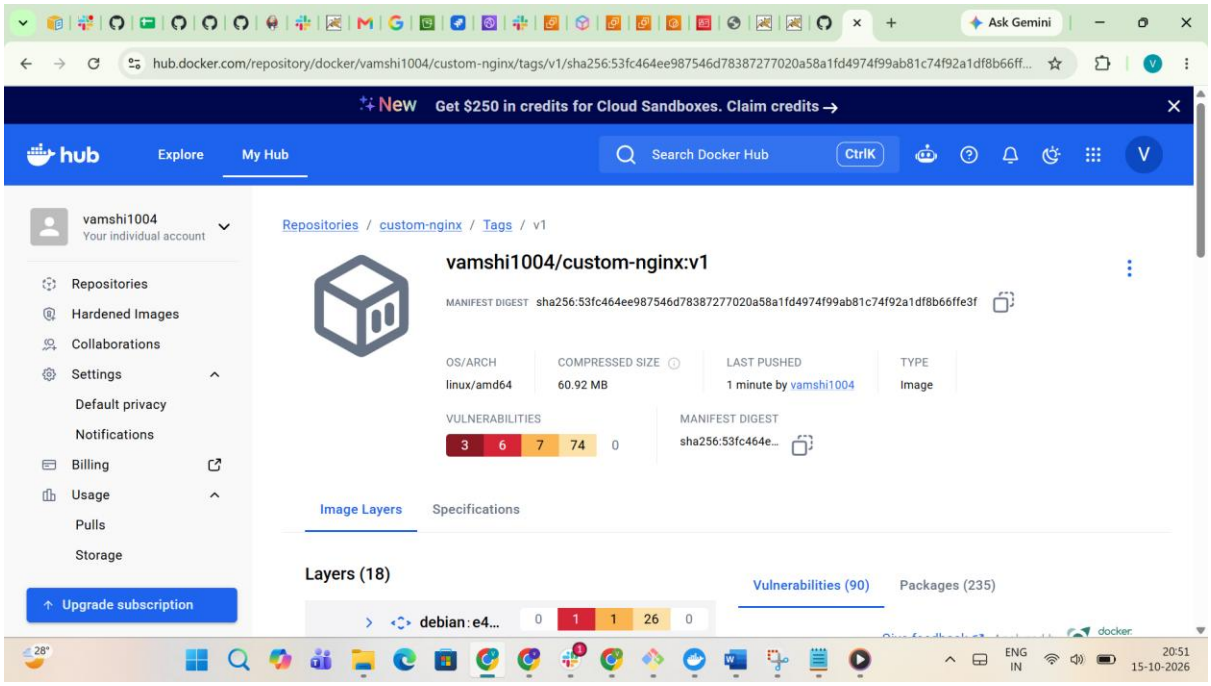
5. Push the image to Docker Hub.

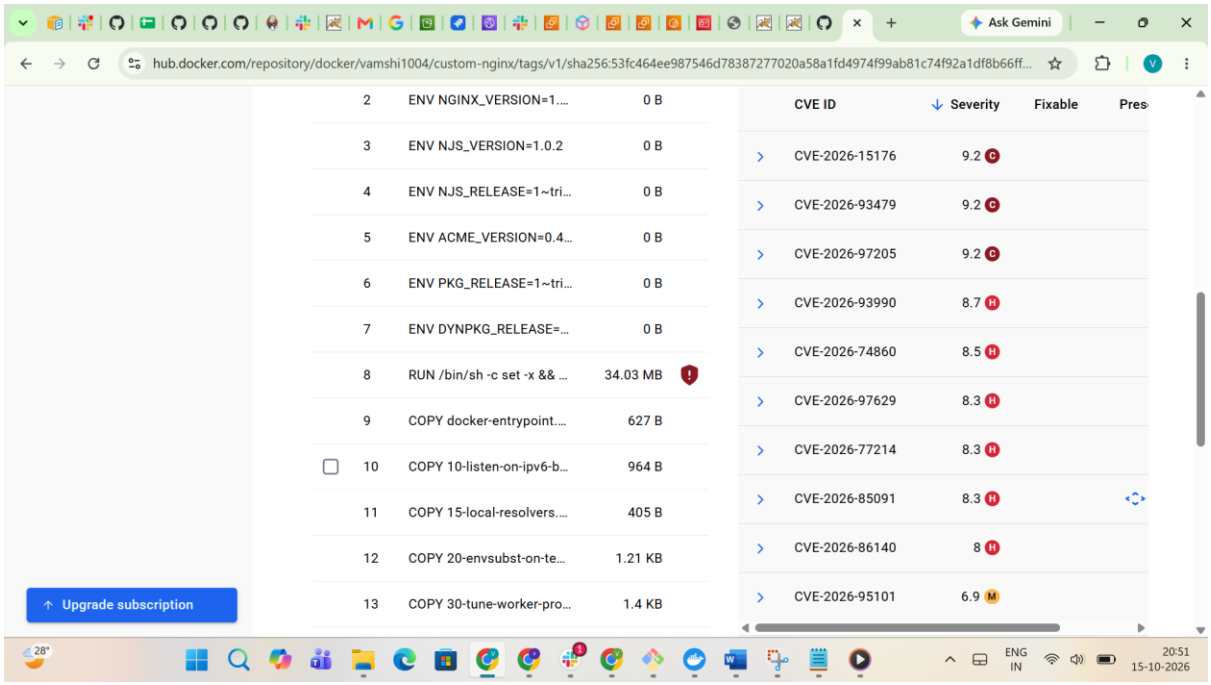
docker push vamshi1004/custom-nginx:v1

Result: Docker images were pushed to Docker Hub and scanned for known software vulnerabilities.









7. Implement solution to scan images when pushed to AWS ECR.

Implementation steps

1. Verified the AWS CLI installation.

   aws --version

2. Configured an IAM role named EC2-ECR-Push-Role for the EC2 instance and attached the AmazonEC2ContainerRegistryPowerUser policy.

3. Verified the active AWS identity.

   aws sts get-caller-identity

4. Created a private ECR repository named custom-nginx in the Mumbai Region (ap-south-1) using the AWS Console.
5. Set the AWS Region and retrieved the account ID.

   REGION=ap-south-1

   ACCOUNT_ID=$(aws sts get-caller-identity \

   --query Account --output text)
6. Authenticated Docker to the ECR registry using the AWS CLI.

   aws ecr get-login-password --region ap-south-1 | \

   docker login \

    --username AWS \

    --password-stdin \

   281712468655.dkr.ecr.ap-south-1.amazonaws.com
7. Tagged the existing custom Nginx image with the ECR registry URI.

   docker tag custom-nginx:latest \

   281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest
8. Pushed the image to the ECR repository.

   docker push \

   281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest
9. Verified the uploaded image using the AWS CLI.

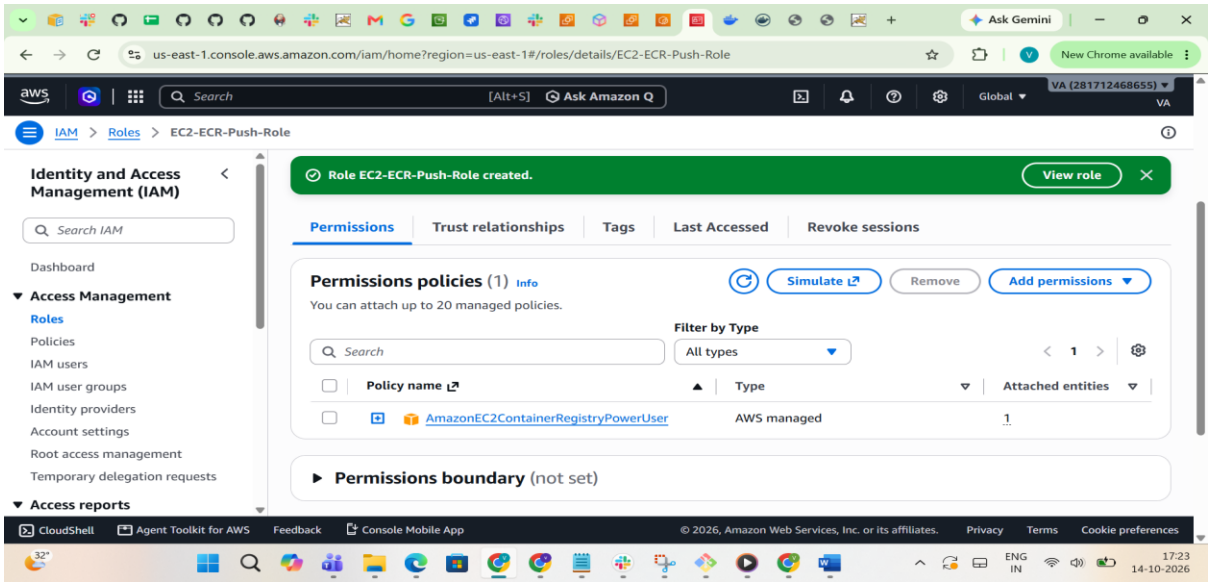
   aws ecr describe-images \

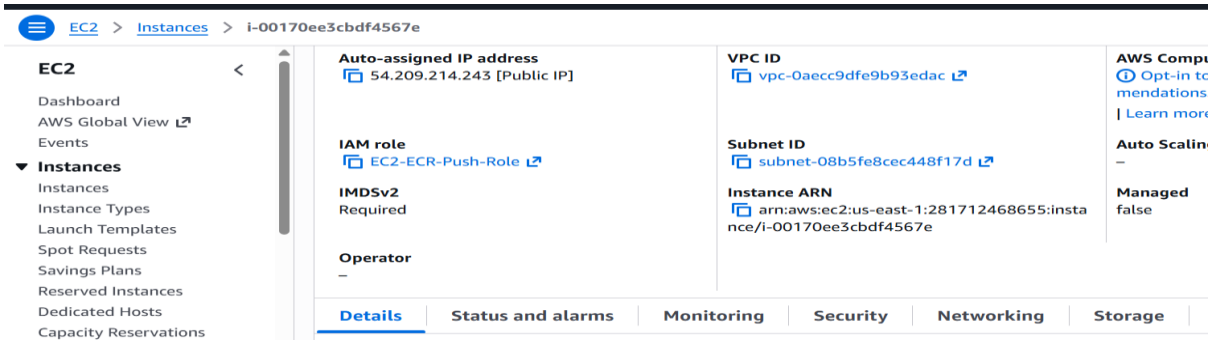
    --repository-name custom-nginx \

   --region ap-south-1
10. Opened the AWS Console and navigated to **Amazon ECR → Private registry → Repositories → custom-nginx → Images** to verify the uploaded latest image tag.

Verify scan findings

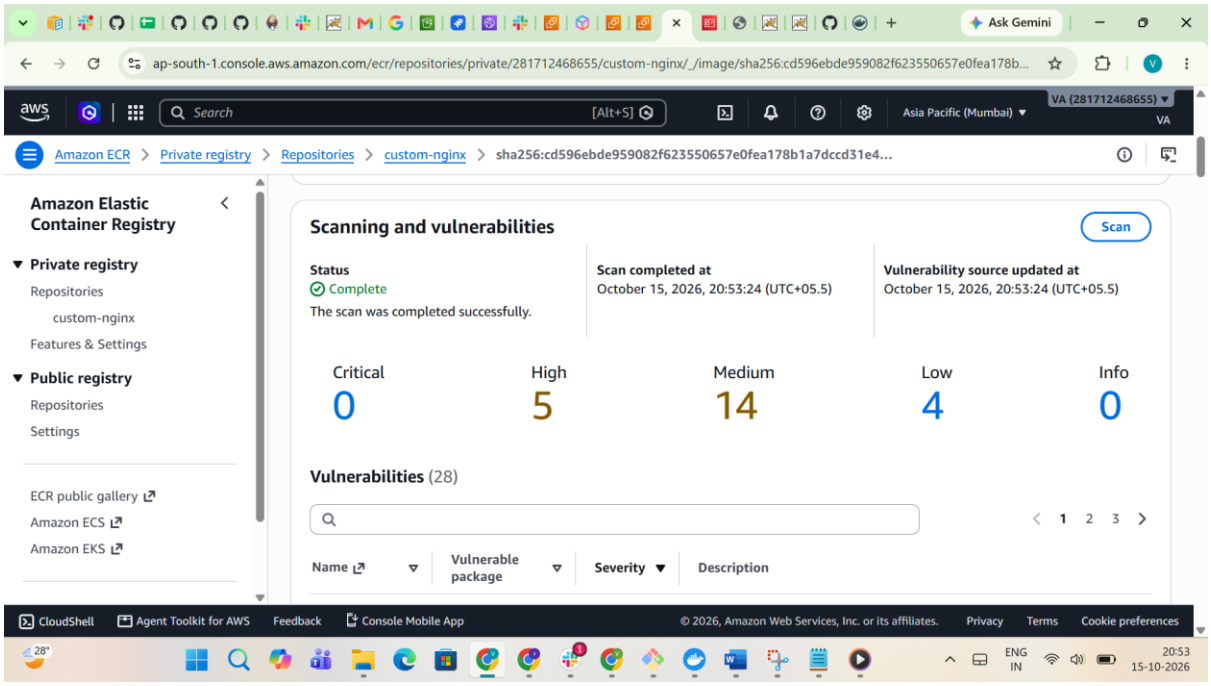
11. Open Amazon ECR → Repositories → multi-stage-example → Images.
12. Select the pushed image.
13. Review its scan status and findings. For enhanced scanning, review the related findings in Amazon Inspector.
14. If vulnerabilities are detected, update the image or dependencies, rebuild, and push a corrected image.






```
[root@ip-172-31-30-23 ~]# aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin \
281712468655.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-30-23 ~]# docker images
REPOSITORY                                                TAG       IMAGE ID       CREATED             SIZE
281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx   latest    1a4e4ef27191   About an hour ago   235MB
custom-nginx                                              latest    1a4e4ef27191   About an hour ago   235MB
vamshi1004/custom-nginx                                   latest    1a4e4ef27191   About an hour ago   235MB
tomcat                                                    latest    6c3d8e81a041   37 hours ago        444MB
nginx                                                     stable    3a8edbda21b6   8 days ago          163MB
[root@ip-172-31-30-23 ~]# docker tag custom-nginx:latest \
281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest
[root@ip-172-31-30-23 ~]# docker push \
281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest
The push refers to repository [281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx]
595572d4bfd9: Pushed
7695e881d922: Pushed
37ab5dcca5fc: Pushed
latest: digest: sha256:cd596ebde959082f623550657e0fea178b1a7dccd31e4e0546d5f9dc3912222c size: 948
[root@ip-172-31-30-23 ~]# aws ecr describe-images \
  --repository-name custom-nginx \
  --region ap-south-1
{
    "imageDetails": [
        {
            "registryId": "281712468655",
            "repositoryName": "custom-nginx",
            "imageDigest": "sha256:cd596ebde959082f623550657e0fea178b1a7dccd31e4e0546d5f9dc3912222c",
            "imageTags": [
                "latest"
            ],
            "imageSizeInBytes": 86762510,
            "imagePushedAt": "2026-10-14T11:50:59.226000+00:00",
            "imageManifestMediaType": "application/vnd.docker.distribution.manifest.v2+json",
            "artifactMediaType": "application/vnd.docker.container.image.v1+json",
            "imageStatus": "ACTIVE"
        }
    ]
}
[root@ip-172-31-30-23 ~]#
```



8. Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

Implementation steps

The Jenkins container is configured on port 8084 and has access to the host Docker engine. The pipeline uses your repository and the Dockerfile.multi content from the previous task.

A. Configure credentials

1. Open Jenkins at http://<EC2_PUBLIC_IP>:8084.
2. Navigate to Manage Jenkins → Credentials → System → Global credentials.
3. Add a credential of type Username with password.
4. Enter your Docker Hub username and an access token.
5. Set the credential ID to dockerhub-creds.

B. Create the pipeline

1. Create a new Jenkins item named DockerHub-Pipeline.
2. Select Pipeline.
3. In the Pipeline section, choose Pipeline script.
4. Add the following script:

```
pipeline {
    agent any
```

```
    environment {
        DOCKERHUB_REPO = 'vamshi1004/multi-stage-example'
    }

    stages {
        stage('Checkout Source Code') {
            steps {
                git branch: 'master',
                    url: 'https://github.com/betawins/multi-stage-example.git'
            }
        }

        stage('Create Multi-Stage Dockerfile') {
            steps {
                sh '''
                    cat > Dockerfile.multi <<'EOF'
FROM maven:3.9-eclipse-temurin-8 AS builder

WORKDIR /app/source

COPY . .

RUN mvn -Dmaven.repo.local=/tmp/m2 clean package -DskipTests

FROM eclipse-temurin:8-jre-alpine

WORKDIR /app

COPY --from=builder /app/source/target/*.jar /app/app.jar
```

```
EXPOSE 8080

ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "/app/app.jar"]
EOF
            '''
        }
    }

    stage('Build Docker Image') {
        steps {
            sh '''
                docker build \
                 -f Dockerfile.multi \
                 -t "$DOCKERHUB_REPO:$BUILD_NUMBER" \
                 -t "$DOCKERHUB_REPO:latest" .
            '''
        }
    }

    stage('Login to Docker Hub') {
        steps {
            withCredentials([
                usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKERHUB_USER',
                    passwordVariable: 'DOCKERHUB_TOKEN'
                )
            ]) {
                sh '''
                    set +x
                    printf '%s' "$DOCKERHUB_TOKEN" |
```

```
                    docker login \
                      --username "$DOCKERHUB_USER" \
                      --password-stdin
                '''
            }
        }
    }

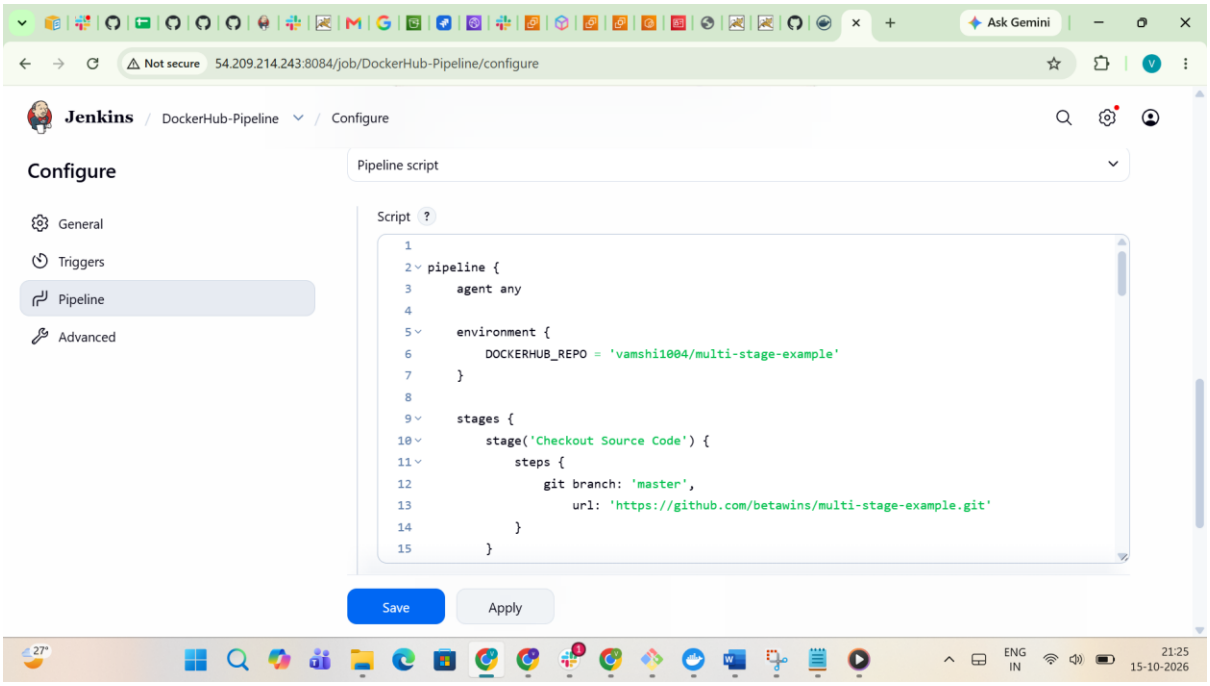
    stage('Push Image to Docker Hub') {
        steps {
            sh '''
                docker push "$DOCKERHUB_REPO:$BUILD_NUMBER"
                docker push "$DOCKERHUB_REPO:latest"
            '''
        }
    }
}

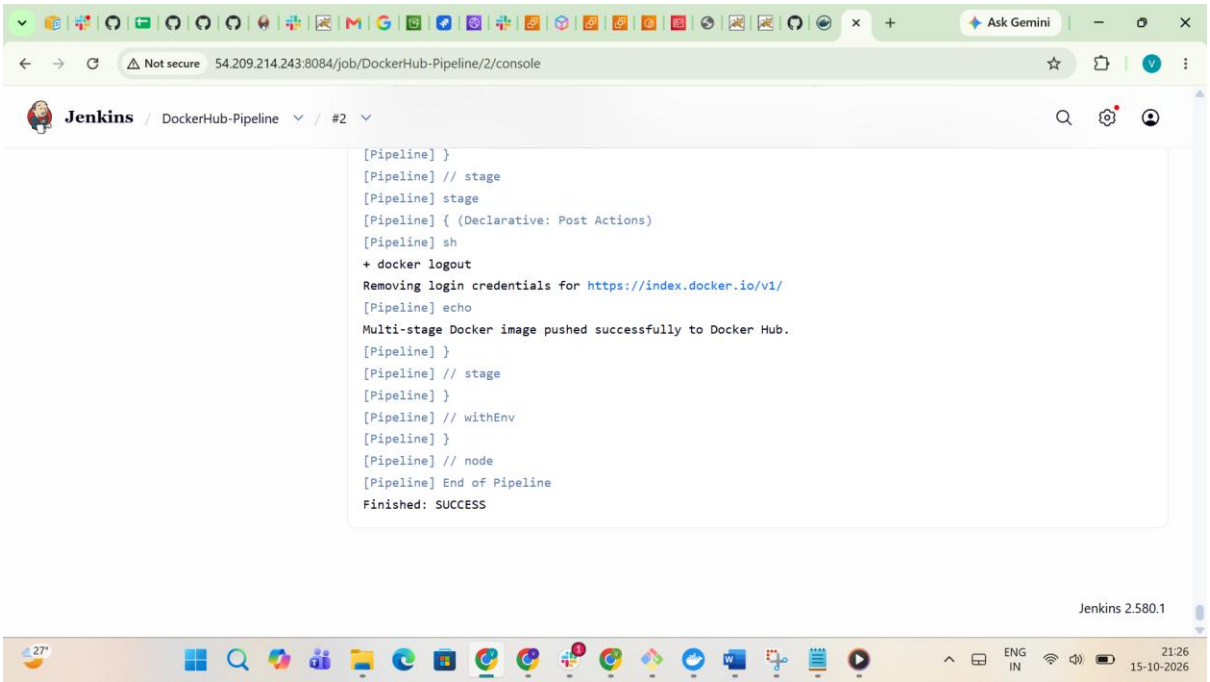
post {
    success {
        echo 'Docker image pushed successfully to Docker Hub.'
    }
    failure {
        echo 'Pipeline failed. Check Console Output.'
    }
    always {
        sh 'docker logout || true'
    }
}
```

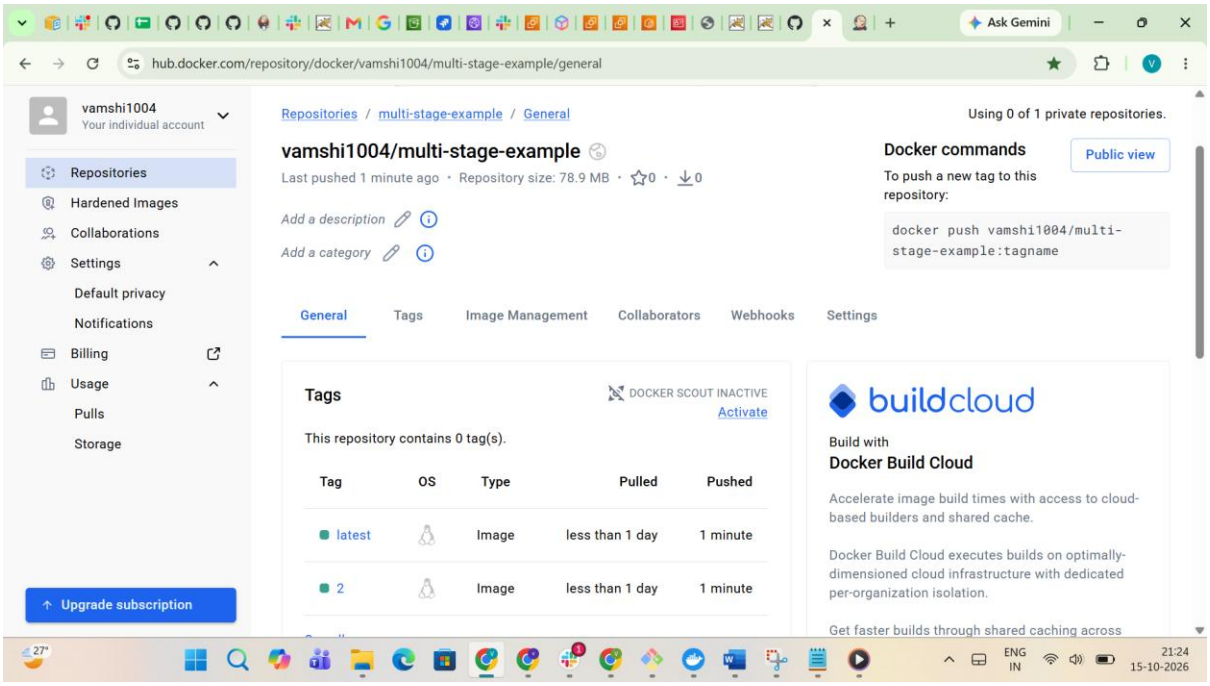
5. Click Save, then Build Now.

6. Open Console Output and verify that checkout, Dockerfile creation, image build, Docker Hub login, and image push complete successfully.
7. Open the Docker Hub repository and confirm that both the build-number tag and latest are present.

Result: A Jenkins pipeline was implemented to retrieve the application source code, create a multi-stage Dockerfile, build the image, and push it to Docker Hub






Ask Gemini
hub.docker.com/repository/docker/vamshi1004/multi-stage-example/general
vamshi1004
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / multi-stage-example / General
Using 0 of 1 private repositories.
vamshi1004/multi-stage-example
Last pushed 1 minute ago • Repository size: 78.9 MB • 0 • 0
Add a description
Add a category
Docker commands
Public view
To push a new tag to this repository:
docker push vamshi1004/multi-stage-example:tagname
General
Tags
Image Management
Collaborators
Webhooks
Settings
Tags
DOCKER SCOUT INACTIVE
Activate
This repository contains 0 tag(s).
Tag
OS
Type
Pulled
Pushed
latest
Image
less than 1 day
1 minute
2
Image
less than 1 day
1 minute
buildcloud
Build with
Docker Build Cloud
Accelerate image build times with access to cloud-based builders and shared cache.
Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.
Get faster builds through shared caching across
27°
ENG
IN
21:24
15-10-2026